

ScamShield

Indian Digital Scam Intelligence & Prevention System

Complete Project Documentation — every technology, working flow, program flow, usage guide, and start-to-end development progress, organized.

Version reviewed : 0.3.0 (post final audit)

Audit status : SCAMSHIELD FINAL AUDIT COMPLETE — 472 tests passing

Mode : 100% local · offline · deterministic · explainable · free (no paid APIs)

Documentation set: this PDF + DOCX + visual PPT in process_threat_detection/

1. Project Overview

ScamShield is a research prototype that analyzes suspicious digital content — SMS/chat messages, embedded URLs, QR codes and UPI payment strings — and returns a 0–100 risk score, a risk level (LOW / MEDIUM / HIGH / CRITICAL), a scam category (e.g. Fake KYC, UPI payment scam, phishing), human-readable explanations with concrete evidence, and safety recommendations.

It is NOT a black-box binary danger detector. It explains why something was flagged so a user can take informed action, and it never executes or contacts anything it analyzes: no internet, no DNS, no URL opening, no payment execution, no paid APIs.

- Scope: Indian digital-scam patterns across SMS, WhatsApp/Telegram, malicious URLs, QR codes and UPI payment strings.
- Unique focus: multilingual support (English, Hindi, Marathi, Hinglish code-mixing) plus explainable, deterministic risk scoring.
- Delivery: final Flask web command center + REST API, research/evaluation lab, and a fully local ML layer.
- Engineering bar: 472 automated tests passing; security-focused (static-only, XSS-safe UI, temp-file cleanup, no stack-trace leakage).

2. Problem & Motivation

Indian users increasingly encounter scams through multiple channels at once:

- SMS and messaging platforms (WhatsApp, Telegram, etc.)
- Malicious or suspicious URLs embedded inside messages
- QR codes that redirect to fraudulent payment or credential-harvesting pages
- Social-engineering tactics exploiting urgency, fear or impersonation

These scams span multiple Indian languages and code-mixed Hinglish, which limits tools built for clean English or single-language input. ScamShield addresses the combination of Indian scam patterns + multilingual content + Hinglish code-mixing + explainable, actionable results, inside a lightweight, zero-budget, fully-local system.

3. Proposed System & Key Outputs

For every input, ScamShield returns a unified result with:

Field	Meaning
risk_score	Continuous 0–100 heuristic score
risk_level	LOW / MEDIUM / HIGH / CRITICAL
is_suspicious	Boolean suspicion flag
scam_type	Specific scam category (e.g. fake_kyc, phishing, upi_payment)
confidence / confidence_type	Heuristic or model probability — always labelled honestly

Field	Meaning
summary / explanation	Human-readable justification with concrete evidence
indicators	Which signals fired (urgency, brand_impersonation, ...)
recommendations	Actionable safety advice
engine_results	Full original per-engine result preserved for transparency

Errors never crash the system: they are returned as structured results (success=False, error, error_type, status='ERROR', score 0/LOW).

4. Supported Scam Categories

Category	Description
Fake KYC	Messages claiming KYC expiry requiring immediate action
UPI/payment scams	Fraudulent UPI collect requests, fake refund links
Bank impersonation	Messages impersonating banks requesting OTP/PIN
Phishing	Links mimicking legitimate services to steal credentials
Fake customer care	Scammers posing as support staff requesting sensitive info
Job scams	Fake job offers requiring registration fees or personal data
Investment scams	Guaranteed-return or crypto-doubling schemes
Loan scams	Fake pre-approved loans requiring upfront fees

Category	Description
Delivery scams	Fake customs/courier payment demands
Lottery/prize scams	Fake prize notifications requiring processing fees
Social-media impersonation	Impersonating relatives or contacts requesting money
Other social-engineering	Miscellaneous patterns (electricity, SIM, court, etc.)

5. Language Support

Language	Code	Notes
English	en	Full rule coverage
Hindi	hi	Devanagari + Romanized variants
Marathi	mr	Devanagari + Romanized variants
Hinglish	hinglish	Code-mixed English–Hindi handled explicitly

6. Research Direction & Honesty Principles

This is a research investigation — not a claim that the project invented scam detection. The system explicitly documents dataset size, class imbalance, cross-split leakage and every engine limitation. Risk scores are labelled as heuristic (not calibrated); the ML confidence is the model's raw predicted-class probability, never presented as a certified probability.

7. Technologies Used

Tech nolog y	Version / Role	Where it is used
Pytho n	3.14.6 (dev) — core language	Everywhere (engines, ML, web, research, docs)
Flask	3.x — final web frontend / REST API	web/app.py + templates + static
Strea mlit	1.32.x — LEGACY prototype UI (not final)	app/ui.py, a pp/ui_helper s.py
scikit -lear n	1.3.x — TF-IDF + Logistic Regression	src/scamshie ld/ml
joblib	1.3.x — model persistence	models/sca mshield_tf idf.joblib
Open CV (h eadle ss)	4.8.x — local QR decoding	src/scamshie ld/qr/decod e.py
qrcod e	7.3.x — QR fixture generation (tests/demo only)	scripts/gene rate_qr_fixtu res.py
num py	1.24.x — QR fixture/tests	scripts, tests
pytes t	latest — 472 tests across 10 suites	tests/
Vanill a JS / HTM L / CSS	Frontend rendering (no framework)	web/static

Layers: user inputs → ingestion & normalisation (detect_input_type, QR decode) → component engines (NLP, URL, QR/UPI) plus the ML layer → unified analyzer → optional chain correlation → unified result → Flask web command center / API. The Research & Evaluation Lab reads the same JSON artifacts used by the dashboard.

10. Working Flow & Program Flow

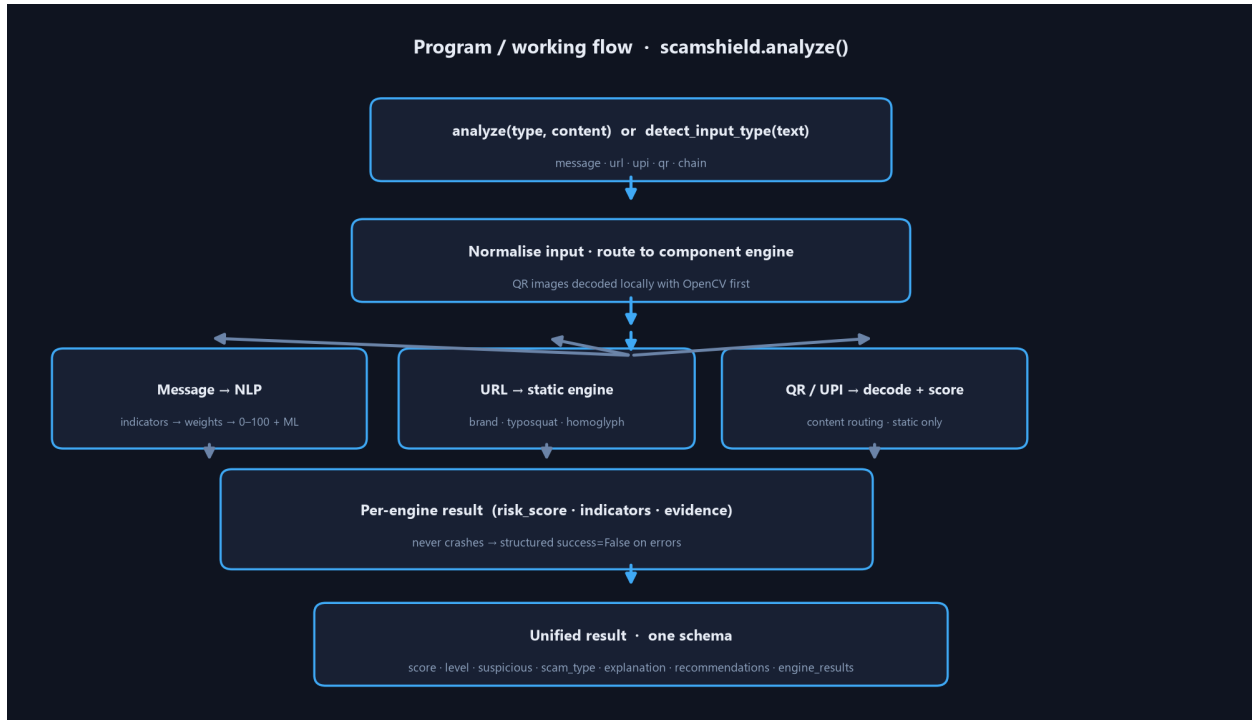


Figure 2 — Program / wiring flow inside `scamshield.analyze()`.

Step-by-step program flow for a single `analyze()` call:

- Caller supplies (type, content) explicitly, or `detect_input_type()` guesses url / upi / message from the prefix.
- Input is normalised (strip, type-check, QR decode).
- Routing: message→NLP; url→URL engine; upi→UPI scorer; qr→decode then route payload; chain→correlation over already-analyzed artifacts.
- Messages get per-URL `url_analysis` + `combined_risk` (always ≥ strongest single signal) + `ml_analysis` (kept separate from the rule verdict).
- Multi-QR images combine risks with the 70% strongest+weaker rule.
- Everything is normalized into one unified schema; failures return `structured success=False` (never crash).

Chain program flow: `normalize_stage()` → correlate pairs (exact URL / root domain / UPI payee / category family) → transparent chain scoring (never a plain average, never below the strongest stage) → pattern detection (`kyc_phishing`, `payment_scam`, `prize_refund`, `fake_customer_care`, `job_scam`, `investment_scam`, `loan_scam`) → classification none / `potential_chain` / `multi_stage_scam`.

11. Component Deep-Dives

11.1 NLP message engine

Rule-based, fully local, deterministic. Pipeline: preprocess → indicator detection from curated multilingual dictionaries → weighted scoring (severity multiplier LOW 1.0 / MEDIUM 1.6 / HIGH 2.2) → family bonuses (family-money +18, credential/transaction guards) → caps → 0-100 score + level + scam type + evidence + recommendations. High-weight signals include `kyc_request` (-10), `otp_request` (12), `credential_request`

(14), suspicious_link (9); ordinary bank-transaction wording is drained (-40) so real receipts stay LOW.

11.2 URL intelligence engine

Static structural analysis only — never opens/resolves/follows/downloads. Detectors: raw IP hosts (+54 with keyword), embedded credentials, punycode/IDN, registrable-domain awareness, brand impersonation with sub-domain placement, typosquatting, Unicode/mixed-script homoglyphs (Cyrillic 'p' in paypal), suspicious query params (password, otp, card), nested/encoded URLs, hex hosts, cheap TLDs, hyphen/numeric-host abuse, unusual schemes — with complexity caps. Also exposes `analyze_attachment_filename()` and `assess_sender()`.

11.3 QR security analyzer

Local decode via OpenCV; payload content-routed url→URL engine, text→NLP, upi→static scorer, unknown→never auto-malicious. Multi-QR combines risks with the 70% strongest+weaker rule; every score point is explained.

11.4 UPI analyzer

Static upi:// parser + scorer: payment_destination_detected +8, embedded_payment_amount +10, amount thresholds 10k/50k/100k, urgency/refund/KYC/OTP note-language. Merchant-pay stays LOW; refund/prize/urgent with high amounts reach HIGH/CRITICAL. Never claims a specific UPI ID is fraudulent — always says verify the recipient.

11.5 Unified analyzer

One schema over message/URL/QR/UPI/chain/ML: `analyze()`, `analyze_batch()`, `detect_input_type()`, `analyze_chain()`. Structured errors, deterministic ordering, full evidence preservation.

11.6 Local ML intelligence

TF-IDF (word+char n-grams) + Logistic Regression, trained offline on data/train/messages.csv. `predict_message()` returns prediction, score (predicted-class probability 0–100), confidence (labelled model_probability, explicitly not calibrated), features_used and model metadata. Missing model → graceful 'ML model unavailable'. The authoritative top-level verdict stays rule-driven; ML is attached separately as `ml_analysis`.

11.7 Chain correlation

Correlates already-analyzed artifacts deterministically. Scoring: stage_bonus decays geometrically, rel_bonus squared+capped at 40, category-consistency bonus ≤8, progress +8, pattern +6; never below the strongest stage. Caveats: correlation is not proof of intent; a lone artifact is never a chain.

11.8 Research & Evaluation Lab

`python -m app.research_eval`: dataset profile, rule/ML/combined metrics, per-language and per-category metrics, sanitised FP/FN examples, detection latency, cross-split leakage audit with a leakage-clean subset. Artifacts → `models/research/` (read live by the web dashboard).

12. Flask Web Command Center (final frontend)

`web/` is a thin HTTP layer: every response comes from the real `analyze()/analyze_chain()` results. No duplicated detection or scoring in Flask or the browser. WSGI-ready (`web.app:app`).

- 8 pages: Overview, Message Intelligence, URL Intelligence, QR Analyzer, UPI Analyzer, Attack Chain, Research/Evaluation, System Status.
- Risk scores/levels come from the backend only; the local ML signal is shown separately from the rule verdict.

- Responsive + reduced-motion respected; all injected data HTML-escaped (no XSS).

Endpoint	Purpose
GET /	Single-page command center (research payload injected server-side)
GET /api/health	Service, version, offline + ML availability
POST /api/analyze/message	{"message": "..."} → rule + local ML
POST /api/analyze/url	{"url": "..."} → static URL analysis (never opened)
POST /api/analyze/upi	{"upi": "..."} → static UPI analysis (never executed)
POST /api/analyze/qr	multipart qr_image → local decode; temp file always cleaned up
POST /api/analyze/chain	{"stages": [...]} → multi-stage chain analysis

QR upload handling: 5 MiB limit, allowed PNG/JPG/JPEG/WebP/GIF/BMP; validated before temp-file write; decoded payloads analysed statically; file unlinked in a finally block (tested).

13. Legacy Streamlit Prototype (kept, NOT final)

app/ui.py was the original prototype frontend (dark cybersecurity theme, SVG gauges, session history in st.session_state with metadata only). Retained for reference and labelled LEGACY — the Flask web app is the final frontend.

14. How to Use

Setup (local, free)

```
python -m venv .venv
pip install -r requirements.txt
```

Run the web app: `python web/app.py` → `http://127.0.0.1:5000`

Use it to: paste a suspicious SMS, check a link, upload a QR image, inspect a UPI string, build a multi-stage attack chain, browse the research dashboard, and check system status.

API

```
POST /api/analyze/message {"message": "Your KYC has expired. Click the link now."} POST
/api/analyze/url {"url": "https://secure-sbi-verify.example.com/login"} POST
/api/analyze/upi {"upi": "upi://pay?pa=refund-center@hdfcbank&am=500000&tn=reward"} POST
/api/analyze/qr multipart qr_image=@file.png POST /api/analyze/chain {"stages": [<full
```

```
analyzed result>, ...]]}
```

CLI / library / training / research

```
python -m app.scan "Your KYC has expired..." python -m app.url_scan
"https://secure-sbi-verify.example.com/login" python -m app.qr_scan
"tests/fixtures/qr/upi_refund_scan.png" python -m app.analyze --type message --input
"Your KYC has expired..." --json from scamshield import analyze, analyze_chain;
analyze("message", "...") python -m src.scamshield.ml.train # retrain local ML (offline)
python -m app.research_eval # evaluation lab → models/research/ python -m pytest tests/
-q # 472 passed
```

15. Start-to-End Development Progress

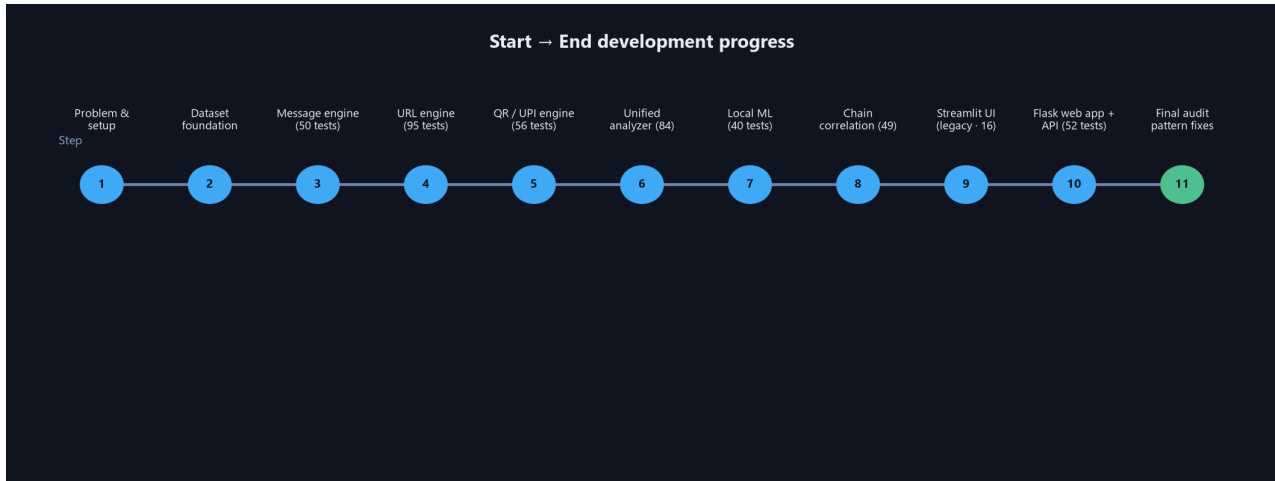


Figure 3 — Milestone timeline (Steps 1–10 + final audit).

Milestone	Delivered	Tests
Step 1	Problem, setup, dataset schema, privacy rules	—
Step 2	Dataset foundation (synthetic + public, scripted splits)	—
Step 3	Multilingual rule-based message scanner	50
Step 4	Static URL intelligence engine (brand/typosquat/homoglyph)	95

M il es to n e	Delivered	Tests
St e p 5	QR / UPI analyzer (decode + routing + static scorer)	56
St e p 6	Unified analyzer (one schema, structured errors)	84
St e p 7	Local ML (TF-IDF + LogReg, leakage audit)	40
St e p 8	Chain correlation layer	49
St e p 9	Streamlit prototype UI (legacy)	16
St e p 10	Flask web command center + API + research dashboard	52
R es e ar c h la b	Full evaluation pipeline + artifacts	30
Fi n al a u di t	Pattern fixes, dep cleanup, research re-run, docs, security + E2E	472 total

Audit highlights: added high-precision patterns (register fee, no experience needed, processing charges, loan, pre-approved) turning paraphrased job scams from 11 to 51 HIGH; removed unused pandas/matplotlib; re-ran the research lab (Rule Acc 0.7736); corrected stale README counts; verified 22 live API checks + full E2E demo over real HTTP with no stack-trace leakage.

16. Evaluation & Research Results

Frozen deduplicated test set n=106 (cross-split leakage reported; a leakage-clean subset is re-evaluated at runtime).

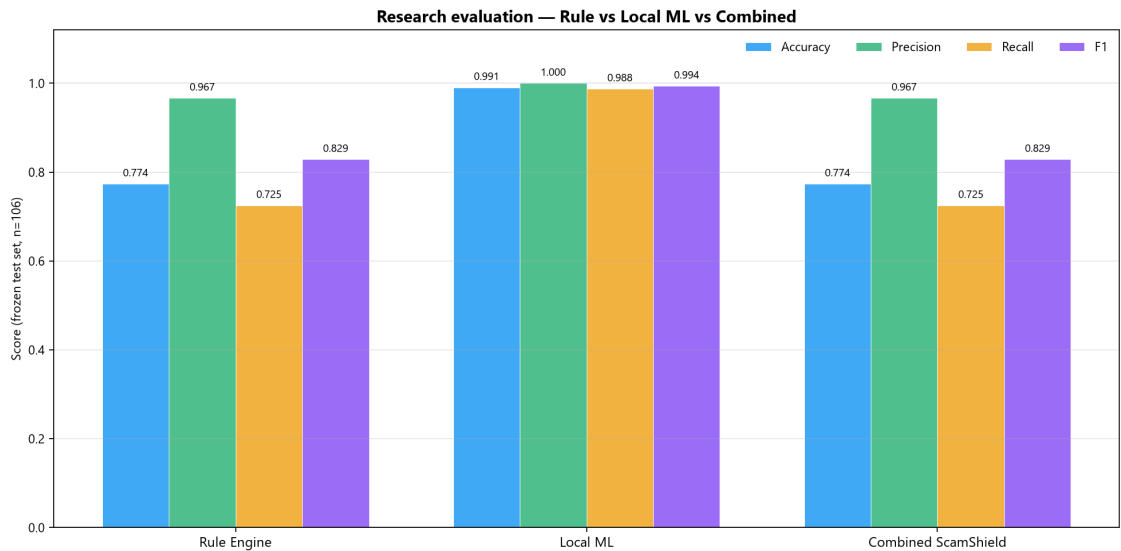


Figure 4 — Accuracy / Precision / Recall / F1 across systems.

System	Accu racy	Preci sion	Reca ll	F1
Rule Engine	0.77 36	0.96 67	0.72 50	0.82 86
Local ML	0.99 06	1.00 00	0.98 75	0.99 37
Combi ned Sc amShie ld	0.77 36	0.96 67	0.72 50	0.82 86

Combined equals Rule on message inputs by design: the deterministic rule engine is authoritative; the ML signal is shown separately as ml_analysis and never averaged in, so a weak or missing ML prediction can never hide a real signal.

Rule-engine misclassification: 22 false negatives (missed scams — the higher-risk error) and 2 false positives; representative examples are persisted, sanitised, in fp_fn_examples.json.

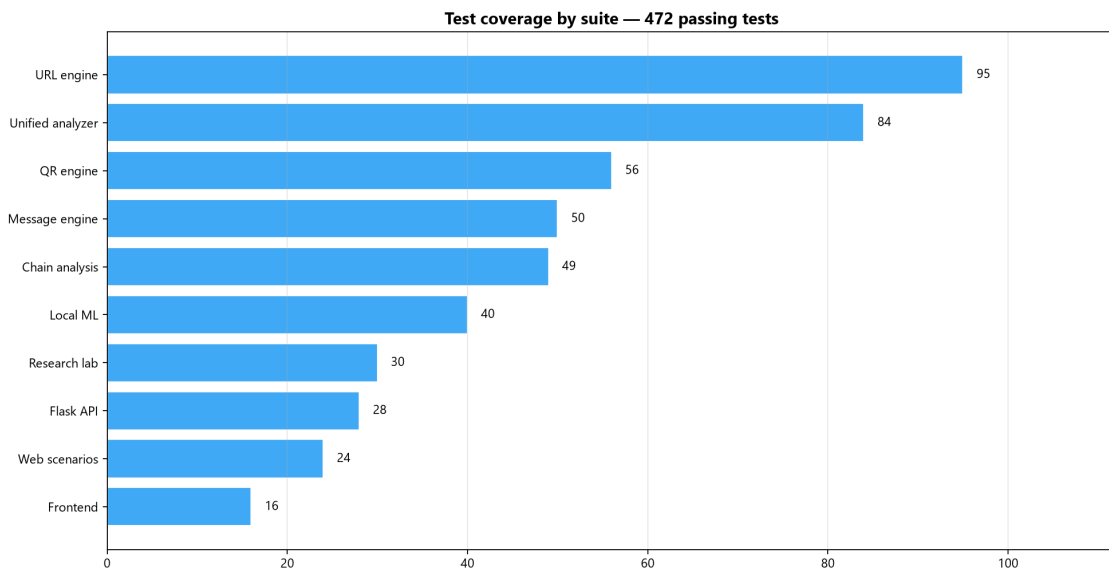


Figure 5 — Test coverage by suite (472 tests).

Latency (20 iterations)	Mean	Media n	Max
message	≈ 28 ms	≈ 28 ms	≈ 31 ms
url	≈ 1.8 ms	≈ 1.6 ms	≈ 4.3 ms
upi	≈ 0.25 ms	≈ 0.15 ms	≈ 2.4 ms
qr	≈ 23 ms	≈ 23 ms	≈ 27 ms

17. Security & Privacy Model

- Fully offline: no sockets / DNS / HTTP / urlopen anywhere in the engines (only urllib.parse + Path.resolve).
- URLs and UPI strings analysed statically — never opened, resolved, followed, fetched or executed.
- QR uploads: size + extension validated, temp file deleted in a finally block (tested).
- No stack traces leaked: 400/404/405/413/500 handlers return structured JSON; debug off by default.
- Web UI HTML-escapes every injected value before innerHTML (no XSS, verified by audit).
- No secrets, API keys, credentials or private data stored or logged anywhere.
- Dataset privacy: placeholders only ([OTP], [PHONE], [ACCOUNT], XX12, *.example.in, RFC 5737 IPs).
- No database, no authentication, no cookies — stateless, local-first, WSGI-hostable.

18. Honest Limitations

- Exact-phrase rule engine: heavily paraphrased scam variants can score LOW/MEDIUM (documented conservative misses).
- Small known false-positive pair: protective 'do not share UPI PIN'-style receipts can be flagged.

- Chain layer requires already-analyzed artifacts (the UI does this); correlation is not proof of intent.
- Local ML is a binary classifier on a small synthetic dataset with class imbalance / leakage — not a production claim.
- URL/UPI analysis is static: no live reputation, DNS or threat-intel (by design — offline, no paid APIs).
- Latency/accuracy figures are from the local research lab, not production traffic.

19. Final Status & Verification

Area	Status
Full test suite	472 passed (10 suites); compileall clean
Flask API (live)	22/22 checks pass incl. malformed / oversized / no trace leakage
E2E demo (real HTTP)	safe LOW · fake-KYC HIGH (+ML 82%) · phishing URL HIGH 72 · refund-UPI HIGH 68 · suspicious QR HIGH 60 · KYC chain CRITICAL 100
Engines	message / URL / QR / UPI / unified / chain / ML all verified
Security	no network/DNS; static-only; temp cleanup; XSS-escaped UI; no secrets
Research artifacts	regenerated and internally consistent with the shipped engine
Docs	READMEs corrected (472, Web UI final); stale counts removed
Dependencies	unused pandas/matplotlib removed; numpy annotated

The project is ready for demonstration as a local, offline, explainable Indian scam & threat-link detection prototype.

20. Appendices

Appendix A — unified result (message example, trimmed)

```
{ "success": true, "input_type": "message", "status": "ANALYSED", "risk_score": 56, "risk_level": "HIGH", "is_suspicious": true, "scam_type": "fake_kyc", "confidence": 66.0, "confidence_type": "heuristic", "summary": "High-risk message classified as a fake kyc scam. Evidence: urgency pressure.", "indicators": ["urgency", "kyc_request", "suspicious_link"], "engine_results": {"message": {...}, "url_analysis": {...}, "ml_analysis": {...}} }
```

Appendix B — API error contract

Validation failures return HTTP 400 with {success:false, error_type:'validation', error: <message>, risk_score:0, risk_level:'LOW'}; 404 unknown routes, 405 wrong methods, 413 oversized uploads, 500 generic internal error (no traceback).

Appendix C — project inventory (key files)

File	Purpose
src/scamshield/nlp/patterns.py	Multilingual indicator dictionaries
src/scamshield/nlp/scoring.py	Weighted 0–100 scoring + caps
src/scamshield/url/indicators.py	Phishing signal detectors
src/scamshield/qr/decode.py	OpenCV QR decoding
src/scamshield/qr/upi.py	Static UPI parser + scorer
src/scamshield/analyzer.py	Unified analyzer
src/scamshield/ml/train.py + predict.py	Offline ML training + prediction
src/scamshield/chain/*	Chain correlation + scoring
src/scamshield/research/*	Evaluation lab + artifacts
web/app.py + templates + static	Final Flask frontend
tests/*	10 suites, 472 tests
scripts/*	Dataset builder, QR fixtures, training
models/research/*	Research evaluation artifacts

Documentation generated from the ScamShield repository — every technology, working flow, program flow, usage guide and start-to-end progress, in one organised document.